Functional Requirements

- Log menstrual periods
- Predict next period
- Predict ovulation date range
- Track daily symptoms and notes
- Provide cycle insights and analytics

Non-Functional Requirements

- Support scalability for millions of users
- Ensure low API response times
- Secure sensitive health data
- Maintain high availability and reliability

Tech stack

Frontend: VueJS 3, Typescript
Backend: ASP.NET Core Web API (.NET)
Database: MySQL
Caching: Redis
MySQL was chosen for its simplicity, relational consistency, and familiarity for rapid implementation. Redis is used to cache frequently accessed prediction data and reduce repeated calculations.

Features to Improve User Experience

The application is designed not only to track menstrual cycles but also to provide meaningful health insights and a personalized user experience. Since menstrual cycles can vary significantly between users, the system supports both regular and irregular cycles while maintaining prediction transparency through confidence scoring. Features such as symptom tracking, ovulation prediction, reminders, and cycle analytics help improve user engagement, usability, and health awareness.

Feature	Benefit	Challenges
Irregular cycles	Better planning	Irregular cycles
Symptom tracking	Health insights	Flexible schema
Notifications (Missed periods)	Better engagement	Scheduling scale
Calendar visualization	Better UX	Sync complexity
Confidence score	User trust	Accuracy calculation

One of the key user experience improvements in the application is the use of confidence-based predictions and irregular cycle handling. Instead of assuming that every user has a regular monthly cycle, the prediction engine adapts based on historical data, cycle variability, and missing periods. This makes the predictions more realistic and trustworthy. Additionally, daily symptom tracking allows users to better understand recurring health patterns and provides opportunities for future personalized insights and analytics.

Scalability Challenges and Solutions

As the application scales to a large number of users, several performance, storage, and reliability challenges need to be considered. The architecture is designed to support scalable API handling, optimized database operations, efficient prediction processing, and secure handling of sensitive health data. Technologies such as caching, asynchronous processing, and database indexing help ensure good performance and responsiveness at scale.

Challenge	Solution
High traffic	Horizontal scaling
Heavy DB reads	Redis caching
Expensive predictions	Async jobs like queue services(SQS)
Notification scaling	Queues/workers
Analytics queries	Pre-aggregations
Large datasets	Partitioning
Sensitive data	Encryption/auth

Prediction generation and dashboard loading can become expensive as the number of users and historical records increase. To address this, frequently accessed prediction data can be cached using Redis to reduce repeated database queries and recalculations. Similarly, background jobs and asynchronous processing can be used for notifications, analytics generation, and prediction updates, helping the system remain responsive even under high traffic conditions. Since the application handles sensitive medical information, security measures such as JWT authentication, HTTPS, password hashing, and encrypted storage are also important considerations as the system grows.

Future Enhancements:

The prediction engine can be extended to support forecasting multiple future cycles instead of only the next predicted period. This can be implemented by recursively using the previously predicted cycle as the input for generating the next prediction. Since long-range predictions may become computationally expensive and less accurate, future cycle predictions can be generated on-demand only when requested by the client application.

For example, once the next predicted period is calculated, the predicted start date can be reused as the input for generating the following cycle prediction. This process can continue for N future months while gradually reducing the confidence score for longer prediction ranges.

Prediction Logic

Important Terms/Notes:

- Outliers: The period lengths might vary in case of PCOS or because of other factors
- Menstrual Phase: Actual bleeding days
- Follicular Phase: From Day 1 of period until ovulation
- Ovulation: Egg release
- Luteal Phase: After ovulation until next period
- Fertile Window:
    - 5 days before ovulation
    - ovulation day
    - 1 day after

Important Formulae:

- Cycle Length = Current Period Start − Previous Period Start
- Weighted Average = fracsum(Cycle × Weight) sumWeights
- Next Period = Last Period + Average Cycle Length
- Ovulation Date = Predicted Next Period − 14 days
- Follicular Phase Length = Ovulation Date − Last Period Start

Algorithm

Constants:
DEFAULT_CYCLE_LENGTH = 28 days
DEFAULT_PERIOD_DURATION = 5 days
LUTEAL_PHASE = 14 days
IRREGULAR_THRESHOLD = 60 days
NO_PERIOD_THRESHOLD = 90 days
CYCLE_WEIGHTS= { mostRecent: 0.5, previous: 0.3, older: 0.2 }

Step 1: Fetch User Period History
periods = fetchPeriods(userId)
sort(periods by startDate ascending)

Step 2: Handle First-Time User
if periods.count == 0

- Use onboarding inputs OR medical defaults
- cycleLength = onboardingCycleLength OR DEFAULT_CYCLE_LENGTH
- periodDuration = onboardingPeriodDuration OR DEFAULT_PERIOD_DURATION
- predictedNextPeriod = today + cycleLength
- ovulationDate = predictedNextPeriod - LUTEAL_PHASE

- follicularPhaseLength = cycleLength - LUTEAL_PHASE
- confidence = 0.30

return prediction

Step 3: Check Minimum Data
if periods.count < 2

- use hybrid approach
- combine defaults + limited history

Step 4: Calculate Cycle Lengths

```
cycleLengths = []
for i from 1 to periods.length - 1:
   cycle = daysBetween(periods[i].startDate, periods[i-1].startDate)
   cycleLengths.add(cycle)
```

Step 5: Remove Outliers

```
validCycles = []
for cycle in cycleLengths:
  if cycle <= IRREGULAR_THRESHOLD:
    validCycles.add(cycle)
```

Step 6: Handle Highly Irregular User

```
if validCycles.count == 0
        use DEFAULT_CYCLE_LENGTH
        mark user as irregular
        confidence = 0.40
```

Step 7: Weighted Average Calculation

```
weights = generateWeights(validCycles.count)
weightedSum = 0
totalWeight = 0
forEach cycle:
  weightedSum += cycle * weight
  totalWeight += weight

avgCycleLength = weightedSum / totalWeight
```

Step 8: Predict Next Period

```
lastPeriod = latestPeriod

predictedNextPeriod = lastPeriod.startDate + avgCycleLength
```

Step 9(Future Enhancement): Long Gap Detection

```
daysSinceLastPeriod = today - lastPeriod.startDate
if daysSinceLastPeriod > NO_PERIOD_THRESHOLD
```

- mark cycle as irregular
- reduce confidence
- stop future auto-predictions
- ask user to log new cycle

Step 10: Ovulation Prediction // 14 days before the next period
ovulationDate = predictedNextPeriod - LUTEAL_PHASE

Step 11: Follicular Phase Calculation
follicularPhaseLength = daysBetween(ovulationDate, lastPeriod.startDate)

Step 12: Fertile Window Calculation
fertileWindowStart = ovulationDate - 5 days
fertileWindowEnd = ovulationDate + 1 day

Step 13: Confidence Score Calculation // Will be updated according future data
```
if firstTimeUser:
   confidence = 0.30
else if variation <= 2:
   confidence = 0.90
else if variation <= 5:
   confidence = 0.75
else:
   confidence = 0.50
if irregular:
   confidence -= 0.20
```

Step 14: Final response
```
{
 "predictionType": predictionType,
 "averageCycleLength": avgCycleLength,
 "averagePeriodDuration": avgPeriodDuration,
 "predictedNextPeriod": predictedNextPeriod,
 "ovulationDate": ovulationDate,
 "follicularPhaseLength": follicularPhaseLength,
 "fertileWindow": {
   "start": fertileWindowStart,
   "end": fertileWindowEnd
 },
 "cycleVariation": variation,
 "isIrregularCycle": isIrregular,
 "daysSinceLastPeriod": daysSinceLastPeriod,
 "confidenceScore": confidence
}
```

Note:

Detailed architecture, database relationship, class, sequence, and activity diagrams are attached separately in the Architecture Diagrams PDF document.